CaseHub – Intelligent Courtroom and Hearing Management System

CLOUD COMPUTING

Submitted by

HARITHA M – 230701107
KANIMOZHI R – 230701140
JAGADEESAN I - 23070119

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**COLLEGE MARKETPLACE - A PLATFORM FOR PEER-TO-PEER TRADING OF PREOWNED GOODS TRADING** " is the bonafide work of **HARITHA(230701107),KANIMOZHI(230701140), JAGADEESAN(23070119)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr.M.Ayyadurai
Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

ABSTRACT

CaseHub is a cloud-based intelligent judicial management system developed to simplify and modernize courtroom operations using Azure cloud services and Generative AI technologies. The system provides an efficient platform for managing hearings, room allocation, case records, notifications, and legal document processing through a user-friendly web interface.

The frontend of the system was developed using HTML, CSS, and JavaScript, while the backend was implemented using Node.js and Express.js. Azure SQL Database is used for storing structured data such as hearing schedules and room allocation details. The project also integrates Azure cloud technologies including Azure App Service for deployment, Docker for backend containerization, Terraform for infrastructure provisioning, and GitHub Actions for CI/CD pipeline automation.

To enhance intelligence and automation, Azure AI Speech Service is integrated for voice-to-text transcription, allowing spoken courtroom discussions or notes to be converted into text. Additionally, Azure AI Language Service is used for legal document summarization, helping judges and administrators quickly understand lengthy case documents.

The system architecture follows modern cloud computing practices with scalable deployment and automated workflows. CaseHub demonstrates how cloud computing, DevOps, and Generative AI can be integrated to improve judicial workflow efficiency, reduce manual effort, and support smart courtroom management.

This project highlights the practical implementation of Azure cloud services, AI-based automation, and modern software deployment techniques in the legal domain.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**Introduction**	1
1.1	Problem Statement	2
1.2	Objective of the Project	4
1.3	Scope and Boundaries	6
1.4	Stakeholders and End Users	8
1.5	Technologies Used	10
1.6	Organization of the Report	12
2	**System Design and Architecture**	14
2.1	Requirement Summary (Functional & Non-Functional)	15
2.2	Proposed Solution Overview	17
2.3	Cloud Deployment Strategy	19
2.4	Infrastructure Requirements	21
2.5	Azure Services Mapping and Justification	23
3	**DevOps Implementation**	26
3.1	Continuous Integration and Deployment (CI/CD) Setup	27
3.2	Terraform Infrastructure-as-Code (IaC)	29
3.3	Containerization Strategy (Docker)	31
3.4	Kubernetes Orchestration (AKS Deployment, Scaling)	33
3.5	GenAI Integration and Azure AI Service Mapping	35
4	**Cloud Operations and Security**	38
4.1	DevSecOps Integration (CodeQL / SonarCloud / ZAP Scans)	39
4.2	Monitoring and Observability (Azure Monitor / App Insights)	41
4.3	Access Control (RBAC Overview)	43

4.4	Blue–Green Deployment & Disaster Recovery Planning	45
5	**Results and Discussion**	48
5.1	Implementation Summary	49
5.2	Challenges Faced and Resolutions	51
5.3	Performance or Cost Observations	53
5.4	Key Learnings and Team Contributions	55
6	**Conclusion and Future Enhancement**	58
6.1	Conclusion	59
6.2	Future Scope	61
	References	63
	Appendices	65

CHAPTER 1 - INTRODUCTION

PROBLEM STATEMENT

Traditional judicial and courtroom management systems largely depend on manual processes, resulting in inefficiency, delays, and difficulty in handling legal operations effectively. Courts often face challenges in managing hearing schedules, room allocations, legal documents, and communication between judicial staff. Existing systems lack intelligent automation and cloud-based scalability, making courtroom administration time-consuming and less organized.

The major problems identified are:

- Manual room allocation and hearing management lead to scheduling conflicts and operational delays.
- Large volumes of legal documents are difficult to analyze and summarize manually.
- Traditional systems do not provide AI-based support for transcription and document intelligence.
- Lack of centralized cloud storage makes data management less efficient and less scalable.
- Absence of automated deployment and infrastructure management increases maintenance complexity.
- Existing judicial systems often lack modern DevOps practices such as CI/CD and containerization.
- Managing courtroom workflows manually increases human effort and reduces productivity.
- There is limited integration of cloud computing and Generative AI technologies in judicial management systems.

To address these challenges, the proposed system "CaseHub" aims to provide an AI-powered cloud-based judicial case management platform that automates hearing management, room allocation, legal document summarization, and voice transcription using Azure cloud services and modern software engineering practices.

OBJECTIVE OF THE PROJECT

The main objective of the project is to develop a cloud-based intelligent judicial case management system named **CaseHub** that simplifies courtroom operations through automation, cloud computing, and Generative AI technologies. The system aims to improve efficiency, reduce manual workload, and provide smart assistance for legal case handling.

The objectives of the project are:

- To develop a centralized platform for managing hearings, cases, and courtroom room allocations.
- To automate hearing room booking and scheduling processes to avoid conflicts and delays.
- To integrate Azure SQL Database for secure and structured storage of judicial data.
- To deploy the application using Microsoft Azure cloud services for scalability and reliability.

- To implement containerization using Docker for easy deployment and environment consistency.
- To use Terraform for infrastructure management and cloud resource provisioning.
- To implement CI/CD pipelines for automated build and deployment processes.
- To provide AI-based legal document summarization using Generative AI services.
- To integrate speech-to-text transcription services for converting courtroom audio into text.
- To improve courtroom workflow efficiency, accessibility, and digital transformation through modern

SCOPE AND BOUNDARIES

Scope of the Project

The scope of the project is to develop a cloud-based judicial case management system that assists judges and legal administrators in managing courtroom activities efficiently using Azure cloud services and AI technologies. The system focuses on automating hearing management, room allocation, legal document handling, and intelligent courtroom assistance.
The major scope of the project includes:

- Managing hearing schedules and courtroom allocations through a centralized dashboard.
- Providing separate interfaces for judicial users such as judges and legal administrators.
- Storing structured courtroom and hearing information using Azure SQL Database.
- Deploying the application on Microsoft Azure cloud platform for scalability and accessibility.
- Implementing containerization and CI/CD automation for modern cloud deployment practices.
- Integrating Azure Speech Service for voice-to-text transcription.
- Integrating Azure Language Service for legal document summarization.
- Providing cloud-based infrastructure management using Terraform.
- Improving judicial workflow efficiency through automation and intelligent services.

Boundaries of the Project

Although the system provides several smart courtroom management features, certain limitations and boundaries exist within the current implementation of the project.
The boundaries of the project are:

- The system is mainly designed for courtroom and hearing management purposes only.
- Full legal decision-making and judgement automation are not included in the system.
- Real-time multi-user synchronization and advanced security features are limited in the current version.
- AI summarization and transcription services depend on Azure service availability and subscription limitations.
- The project currently supports basic backend functionalities and prototype-level integration.
- The system does not include complete e-filing or nationwide judicial integration features.
- Deployment and AI features are limited by Azure for Students subscription policies and resource quotas.

- The application is developed primarily for academic and demonstration purposes rather than full-scale production deployment

STAKEHOLDERS AND END USERS

Stakeholders

Stakeholders are the individuals or organizations that are directly or indirectly involved in the development, deployment, and usage of the CaseHub system. They play an important role in defining system requirements, managing operations, and ensuring efficient judicial workflow.

The major stakeholders of the project are:

- Judges – Manage hearings, review case information, allocate hearing rooms, and utilize AI-based summarization and transcription features.
- Court Administrators – Monitor courtroom schedules, manage room allocation, maintain records, and oversee system operations.
- Lawyers and Legal Professionals – Access hearing information, case schedules, and legal document summaries.
- Judicial Institutions – Benefit from improved courtroom management, automation, and digital transformation.
- System Developers and Administrators – Maintain the application, cloud infrastructure, deployment pipelines, and database services.
- Microsoft Azure Cloud Platform – Provides cloud infrastructure, AI services, deployment environment, database hosting, and DevOps support.

End Users

End users are the people who directly interact with the CaseHub application for performing judicial and administrative activities.

The primary end users of the system are:

- Judges
  - Schedule and manage hearings
  - Allocate courtroom rooms
  - Access summarized legal documents
  - Use AI-powered transcription features
- Court Staff and Administrators
  - Manage hearing schedules and notifications
  - Maintain courtroom records and allocations
  - Monitor system workflows
- Lawyers
  - View hearing schedules and case updates
  - Access hearing-related information and notifications
- Legal Support Personnel
  - Assist in document handling and courtroom coordination
  - Utilize cloud-based records and hearing management features

TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

HTML	For designing the structure of the frontend pages
JavaScript	For implementing frontend functionality and interactivity
Express.js	For creating backend APIs and server handling

Table 2: Backend Technologies

Go	Chosen for its concurrency support, speed, and simplicity in building scalable APIs
Gin Framework	lightweight HTTP router
GORM	An ORM library for seamless database interactions and migrations.
PostgreSQL	relational database for structured data storage.
Blob Storage	For storing images

Table 3: Cloud and Infrastructure

Microsoft Azure	The primary cloud provider for hosting and managing services.
Azure Container Apps	For container orchestration with auto-scaling.
Azure PostgreSQL Flexible Server	Managed database for storing structured data.
Azure Storage Account	For blob storage of user-uploaded images.

Azure Functions	Serverless execution for gen-ai description.
Terraform	For IaaC provision
Docker	For containerizing applications.

Table 5: DevOps and CI/CD

GitHub Actions	For automating workflows, builds, and deployments.
Docker Hub	As a registry for storing and distributing container images.
Azure CLI	For command-line management of Azure resources.

ORGANIZATION OF THE REPORT

The project report is organized into multiple chapters to provide a clear understanding of the proposed system, technologies used, implementation process, and outcomes of the project.

- **Chapter 1 – Introduction**
  This chapter introduces the CaseHub system, including the problem statement, objectives, scope, stakeholders, technologies used, and overall organization of the report.
- **Chapter 2 -Literature Survey**
  This chapter discusses existing judicial management systems, cloud-based courtroom solutions, AI-powered legal technologies, and related research studies relevant to the proposed project.
- **Chapter 3 – System Analysis and Design**
  This chapter explains the system architecture, requirement analysis, database design, workflow diagrams, UI design, and overall system planning.
- **Chapter 4 – Implementation**
  This chapter describes the frontend development, backend implementation, Azure cloud deployment, Azure SQL integration, Docker containerization, Terraform setup, and CI/CD pipeline implementation.
- **Chapter 5 – AI Integration and Cloud Services**
  This chapter explains the integration of Azure AI services such as speech-to-text transcription and legal document summarization using Generative AI technologies.
- **Chapter 6 – Testing and Results**
  This chapter presents system testing procedures, output screenshots, deployment results, database connectivity verification, and performance evaluation of the application.
- **Chapter 7 – Conclusion and Future Enhancements**
  This chapter summarizes the project outcomes, advantages of the proposed system,

limitations, and future improvements that can be implemented in the system

CHAPTER 2 – SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

- The proposed system, CaseHub, consists of multiple cloud-based services deployed mainly on Microsoft Azure to provide an intelligent, scalable, and secure judicial case management platform.
- The frontend of the application is developed using HTML, CSS, and JavaScript and is hosted through Azure App Service and GitHub integration. The platform provides an interactive dashboard for judges and court administrators to manage hearings, room allocation, case details, and notifications efficiently.
- The backend services are implemented using Node.js and Express.js. These services handle core functionalities such as hearing management, room booking, notification handling, database communication, and AI service integration. The backend application is containerized using Docker to ensure portability, lightweight deployment, and environment consistency across development and production systems.
- The application data including hearing schedules, room allocation details, judicial records, and user information are stored in Azure SQL Database. This service provides structured relational data storage with secure cloud accessibility and automated backup support.
- For deployment and infrastructure management, Microsoft Azure App Service and Azure Web Services are used to host the application. Terraform is utilized for Infrastructure as Code (IaC), enabling automated cloud resource provisioning and reproducible deployments.
- The project also integrates Azure AI services for intelligent courtroom assistance. Azure AI Speech Service is used for speech-to-text transcription, allowing courtroom audio and hearing discussions to be converted into text format. Azure AI Language Service is used for legal document summarization, helping judges and legal professionals quickly understand lengthy case documents.
- CI/CD automation is implemented using GitHub Actions integrated with Azure deployment workflows. This enables automated build, testing, and deployment whenever changes are pushed to the GitHub repository.

Non-Functional Requirements

- The system is designed to satisfy important non-functional requirements related to performance, reliability, scalability, maintainability, and security.
- To ensure high performance, the application aims to provide fast dashboard loading and quick response times for hearing management and database operations. The frontend is optimized with lightweight UI design and cloud-based hosting for efficient content delivery.
- The system is designed to support multiple concurrent users such as judges, administrators, and legal staff. Azure cloud infrastructure provides scalability through managed services and cloud resource allocation.
- Availability and reliability are maintained through Microsoft Azure cloud services, ensuring minimal downtime and continuous access to the judicial platform.

- Security is enforced through authenticated user access, secure database connectivity, protected API communication, and Azure cloud security mechanisms. Sensitive judicial information is securely stored and accessed only by authorized users.
- The system architecture follows a modular approach, improving maintainability and enabling future feature expansion such as advanced AI analytics, e-filing systems, and real-time courtroom monitoring.
- The project is also designed with cost efficiency in mind by utilizing Azure student subscription services and free-tier cloud resources wherever possible during development and testing stages.
- 2.2 PROPOSED SOLUTION OVERVIEW
- The proposed solution is a cloud-based intelligent judicial management platform designed to simplify courtroom operations through automation, cloud deployment, and Generative AI integration.
- The architecture follows a layered approach consisting of frontend, backend, database, cloud deployment, and AI service integration layers.
- The frontend provides an interactive dashboard where judges and administrators can manage hearings, allocate rooms, monitor notifications, and access case information. The frontend communicates with backend APIs developed using Node.js and Express.js.
- The backend layer handles all business logic including hearing scheduling, room allocation, database operations, notification handling, and AI service communication. These backend services are containerized using Docker and deployed on Microsoft Azure cloud infrastructure.
- Azure SQL Database serves as the primary relational database for storing hearing schedules, user records, room allocation information, and judicial data. The database supports secure cloud-based access and reliable data management.
- The system integrates Azure AI services to provide intelligent courtroom assistance. Azure AI Speech Service converts courtroom audio into text transcripts, while Azure AI Language Service summarizes lengthy legal documents into concise information for faster review.
- Terraform is used to provision cloud infrastructure resources automatically, improving deployment consistency and reducing manual cloud configuration efforts.
- GitHub Actions automates the CI/CD workflow by continuously building, testing, and deploying the application whenever updates are made to the repository.
- This architecture combines cloud computing, DevOps practices, and Generative AI technologies to create a modern judicial assistance platform.

2.3 CLOUD DEPLOYMENT STRATEGY

- The deployment strategy uses Microsoft Azure as the primary cloud platform for hosting, database management, AI integration, and infrastructure deployment.
- The frontend application is deployed through Azure App Service integrated with GitHub repositories for automated deployment and version management. The backend Node.js services are containerized using Docker and deployed within Azure cloud services for efficient execution and portability.
- Azure SQL Database is used for secure relational data storage with backup support and cloud accessibility. Database connectivity is established through secure authentication and firewall configuration.
- Terraform is used to define and provision Azure cloud infrastructure resources such as

app services, databases, and deployment configurations. This Infrastructure as Code approach improves deployment automation and consistency.

- Docker containerization is implemented to package backend services along with their dependencies, ensuring consistent execution across development and deployment environments.
- GitHub Actions handles Continuous Integration and Continuous Deployment pipelines. Whenever code changes are pushed to GitHub, automated workflows trigger application build, testing, and deployment processes.
- Azure AI Speech and Language services are deployed as managed cloud AI services and integrated into the backend application through API-based communication.
- The overall deployment architecture provides scalability, maintainability, cloud accessibility, and simplified deployment management for the judicial platform.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources
Azure App Service for frontend hosting and application deployment.
Node.js backend server running on Azure cloud environment.
Docker containerization for backend deployment consistency.
GitHub Actions runners for CI/CD workflow automation.

Storage Requirements
Azure SQL Database for structured judicial and hearing data storage.
Cloud-based backup support for database reliability.
Secure storage of hearing schedules, room allocations, user details, and case records.

AI Service Requirements
Azure AI Speech Service for voice-to-text transcription.
Azure AI Language Service for legal document summarization.
API integration support for Generative AI features.

Networking and Security Requirements
Secure API communication between frontend and backend services.
Azure firewall and database access control mechanisms.
Authenticated access for judges and administrators.
HTTPS-based secure cloud deployment.

DevOps and Deployment Requirements
GitHub repository for source code management.
Terraform scripts for infrastructure provisioning.
Dockerfile configuration for backend containerization.
GitHub Actions workflow files for automated CI/CD pipelines.

2.1 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	Azure Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost (₹)
Frontend Hosting	Azure Static Web Apps	Standard	Service Responsive UI	Free hosting, SSL, CDN	Free (Student Credit)
Backend Logic & APIs	Azure Container Apps	0.75 vCPU, 1.5GB	Host Containerize d backend	Always-on, scalable	3000
Database	Azure PostgreSQL Flexible Server	1 vCore, 5GB	Store club details, events, and member data	Reliable, auto-backup, high availability	1,200
Media Storage	Azure Blob Storage	Hot Tier, Geo-Redundant Storage	Store profile and product images	Cost Efficient	300
AI Moderation	Azure Content Moderator	Standard	Filter inappropriate content	Improves safety & compliance	0
Monitoring & Logs	Azure Monitor + Application Insights	Standard	Log and track API health	Proactive maintenance	400
AI Description generation	Gemini Pro Vision	Pay-per-use	Auto-generate descriptions	Boosts listing quality	50

CI/CD Automation	Github Actions	Basic	Automated builds & deployments	Seamless GitHub integration	Free (Student Credit)

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses a multi-stage CI/CD pipeline built with GitHub Actions to automate the deployment. This setup uses Infrastructure as Code (IaC) and containerization to deploy components across various Azure services.

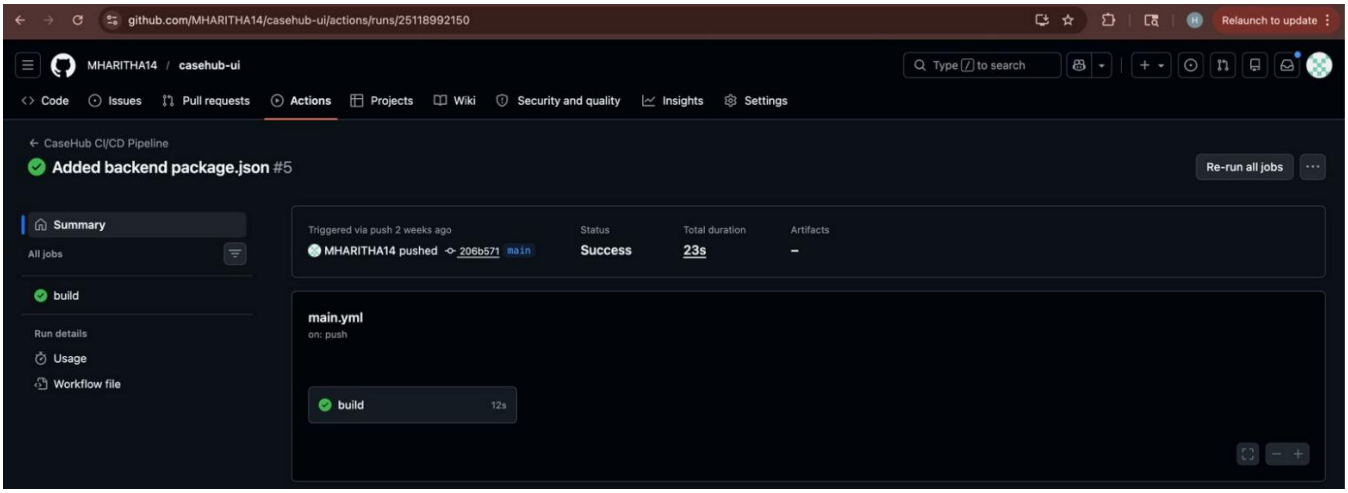


Figure 1. Github Actions Workflow

The pipeline follows 2-stage deployment pattern:

1. **Backend:** Deploys the main application (as a Container App).
2. **Frontend:** Builds and deploys the static web app (React).

The entire process is triggered either by a push to the main branch or through a manual workflow dispatch.

Pipeline Stages in Detail:

Stage 1: Backend Deployment (build-and-deploy-backend)

1. Versioning and Containerization

A unique version tag is generated for every backend deployment using the current date and GitHub commit ID. This helps maintain deployment history and rollback support. Example version format: v20260510-a1b2c3d

The backend application developed using Node.js and Express.js is containerized using Docker. The Docker image packages all backend dependencies and runtime configurations, ensuring consistent execution across all environments.

2. Infrastructure as Code (IaC)

Terraform is used to provision and manage Azure cloud resources required for the application deployment. Infrastructure configurations such as Azure App Service, Azure SQL Database, and deployment settings are defined as code, ensuring reproducibility and simplified cloud management.

Terraform enables:
Automated infrastructure provisioning
Consistent deployment environments
Simplified cloud resource updates
Reduced manual configuration errors

3. Azure Deployment

Secure authentication with Microsoft Azure is handled using Azure credentials and deployment secrets configured within GitHub Actions.
The backend container image is automatically deployed to the Azure cloud environment whenever changes are pushed to the GitHub repository. The deployment process updates the application without affecting existing services, ensuring smooth backend availability during updates.

4. Health Check and Validation

After deployment, automated validation checks are performed to ensure the backend service is functioning correctly.
The CI/CD pipeline verifies:
Successful backend startup
Database connectivity
API availability
Service responsiveness
Only after successful validation does the deployment process complete.

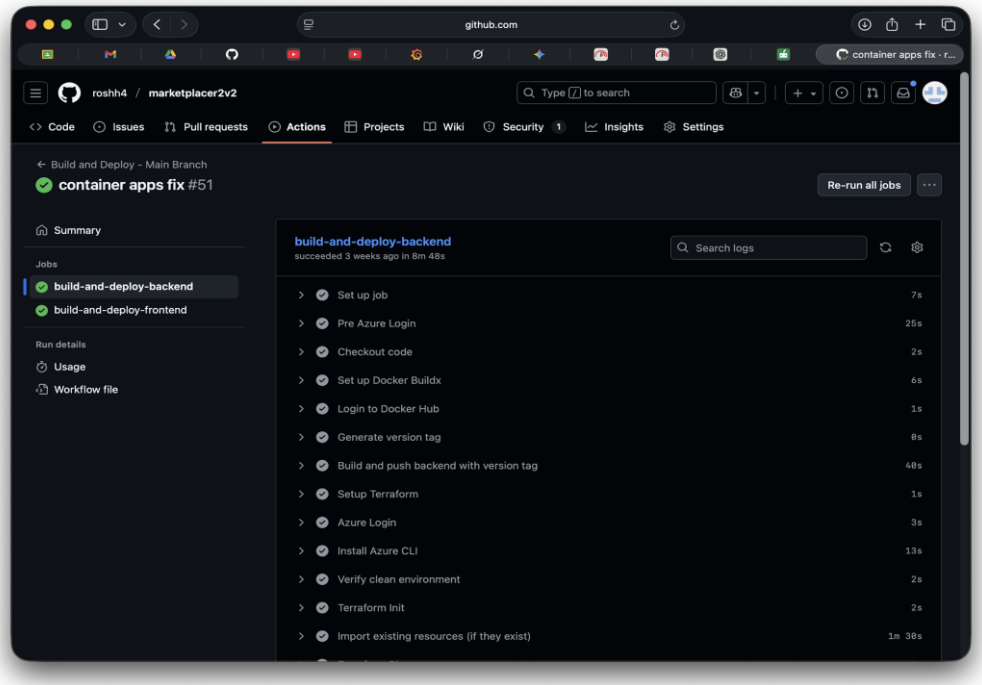


Figure 2. Backend Deployment Job Run

Stage 2: Frontend Deployment (build-and-deploy-frontend)

Dynamic Configuration

The frontend application dynamically connects with the deployed backend services using environment-based configuration. Backend API endpoints are automatically configured during deployment to ensure proper communication between frontend and backend services.

This process ensures:

- Correct API endpoint integration
- Environment-specific configuration handling
- Simplified deployment management

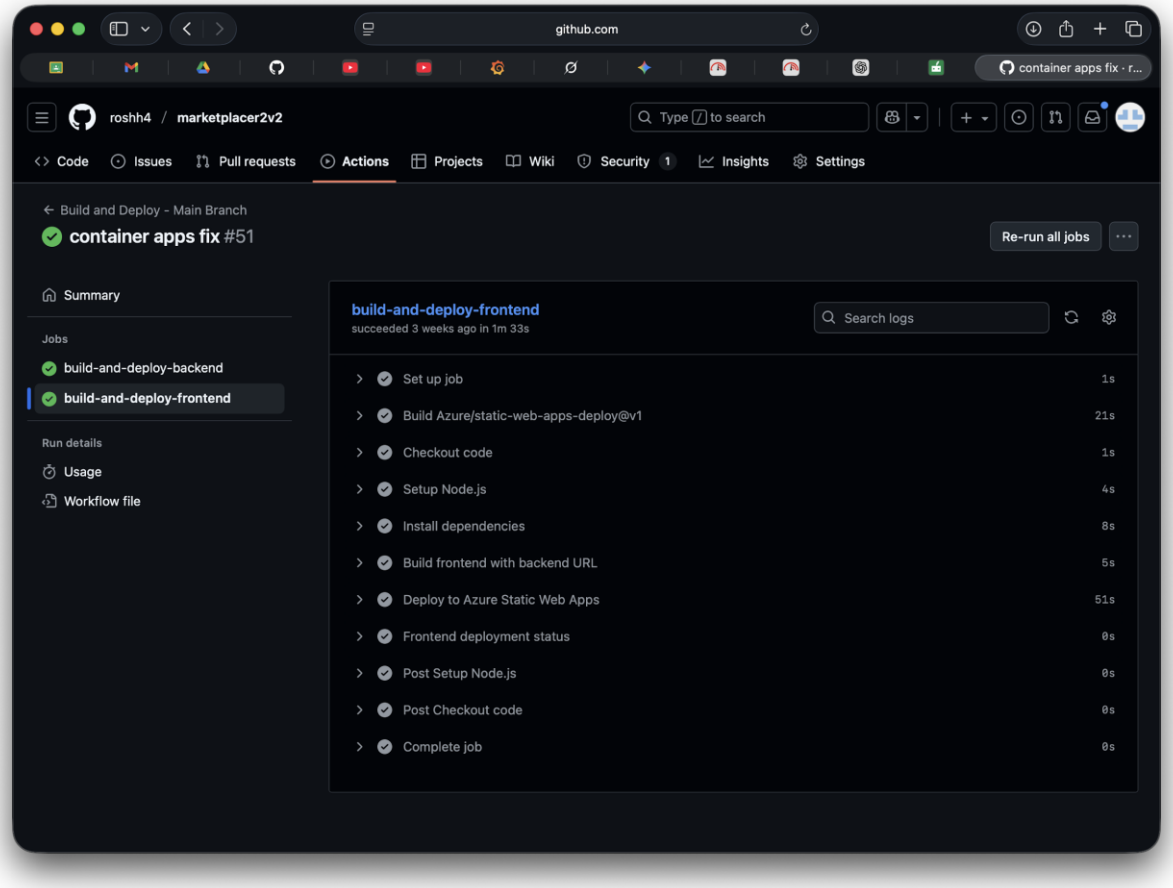


Figure 3. Frontend Deployment Job Run

Required GitHub Secrets

Table 7. List of Github Secrets

1	AZURE_CLIENT_ID
2	AZURE_CLIENT_SECRET
3	AZURE_STATIC_WEB_APPS_API_TOKEN
4	AZURE_STORAGE_ACCOUNT_NAME
5	AZURE_SUBSCRIPTION_ID
6	AZURE_TENANT_ID
7	CONTENT_SAFETY_KEY
8	DB_PASSWORD
9	DOCKERHUB_TOKEN
10	DOCKERHUB_USERNAME
11	GEMINI_API_KEY

The CaseHub project uses Terraform as the Infrastructure-as-Code (IaC) tool to automate the provisioning and management of Azure cloud resources. Terraform helps maintain consistent cloud infrastructure, simplifies deployment, and reduces manual configuration effort.
The Terraform configuration is organized using a standard file structure:
main.tf – Contains the primary definitions and configurations for Azure cloud resources.
variables.tf – Stores configurable input variables such as resource names, database settings, and deployment locations.
outputs.tf – Defines important output values such as application URLs, database server names, and deployment endpoints for integration with CI/CD workflows.

Core Infrastructure Components
The Terraform setup provisions and manages all required Azure cloud resources under a centralized resource group environment for the CaseHub application.

Frontend Hosting Infrastructure
Azure App Service
Terraform provisions Azure App Service resources for hosting the frontend judicial dashboard application.
The App Service configuration includes:
Frontend application hosting
GitHub deployment integration
Secure HTTPS access
Cloud-based scalability support

Backend Infrastructure and Deployment
Backend Application Environment
The backend services developed using Node.js and Express.js are deployed through Azure cloud infrastructure with Docker container support.
Terraform manages:
Backend hosting configuration
Application deployment settings
Environment variables
Cloud networking configurations
Docker Container Deployment
The backend application is packaged using Docker and deployed consistently across environments.
The deployment configuration supports:
Lightweight backend execution
Simplified deployment updates
Environment consistency
Scalable cloud deployment

Database Infrastructure
Azure SQL Database
Terraform provisions Azure SQL Database resources for storing judicial and hearing-related data.
Database configuration includes:

Secure relational database setup
Cloud-based accessibility
Automated backups
Secure authentication mechanisms
The database stores:
Hearing schedules
Room allocation details
User information
Judicial records
Notifications and logs

AI Service Infrastructure
Azure AI Services
Terraform provisions configurations required for integrating Azure AI services such as:
Azure AI Speech Service
Azure AI Language Service
These services provide:
Voice-to-text transcription
Legal document summarization
AI-assisted courtroom support

CI/CD Integration
The Terraform infrastructure is fully integrated with GitHub Actions-based CI/CD workflows.
During CI/CD execution:
Sensitive deployment credentials and Azure configuration values are securely passed through GitHub Secrets.
Updated backend container versions are automatically deployed during pipeline execution.
Infrastructure outputs such as application URLs and database connection details are generated through outputs.tf and reused in deployment workflows.
This automation ensures reliable and repeatable cloud deployment processes for the CaseHub platform.

3.2 CONTAINERIZATION STRATEGY (DOCKER)

The CaseHub project uses Docker containerization to package backend services into portable and consistent deployment units. Docker simplifies deployment, improves scalability, and ensures the backend application behaves consistently across different environments.

Image Build Process
The backend application developed using Node.js and Express.js is containerized using Docker.
The Docker build process includes:

- Builder Stage
- The application source code and required dependencies are copied into the Docker environment.
- This stage performs:
- Dependency installation
- Backend configuration setup

- Application preparation for deployment

Runtime Stage
The final Docker image contains only the required runtime components necessary to execute the backend application.
This approach provides:
Lightweight container images
Improved deployment speed
Reduced storage usage
Better security through minimized dependencies

Deployment and Scaling
The Docker containerized backend is deployed to Microsoft Azure cloud services.
The deployment strategy supports:
Simplified backend deployment
Scalable cloud execution
Faster deployment updates
Improved application portability

Versioning Strategy
Each backend deployment is associated with a unique container version tag generated during CI/CD execution.
Example:
v20260510-a1b2c3d
This versioning mechanism helps:
Track deployment history
Manage application updates
Support rollback procedures
Maintain deployment consistency

Benefits of Docker Containerization
The Docker-based deployment strategy provides several advantages:
Consistent backend execution across environments
Simplified cloud deployment
Faster CI/CD integration
Improved scalability and maintainability
Reduced dependency conflicts
Better deployment portability

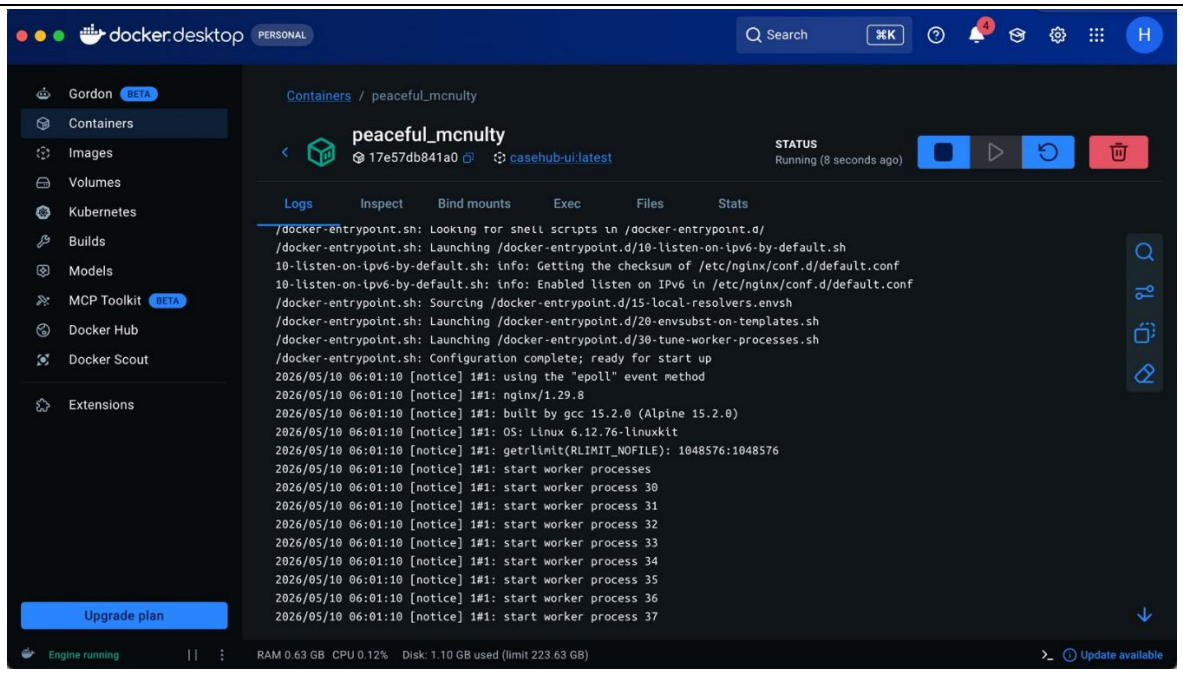


Figure 4. Docker file

1. Azure Deployment Setup

The CaseHub application is deployed on Microsoft Azure cloud infrastructure to provide secure, scalable, and reliable judicial management services.

Resource Allocation

The backend services are configured with optimized cloud resource allocation to support efficient execution and cost-effective deployment.

The deployment configuration includes:

- Backend application hosting through Azure cloud services
- Optimized compute and memory allocation
- Cloud-based application accessibility

Application Scalability

The deployment architecture is designed to support scalability based on application usage and workload demands.

The system supports:

- Dynamic cloud resource scaling
- Multiple concurrent users
- Reliable hearing and room allocation management
- Efficient backend execution under varying workloads

Secure Access and Networking

External access to the application is secured through HTTPS-based communication.

Security measures include:

- TLS/HTTPS secure communication
- Protected API endpoints
- Secure cloud deployment configuration
- Controlled database connectivity

2. Container Image Versioning

Docker images for the backend application are automatically built and managed through GitHub Actions CI/CD pipelines.

A structured versioning strategy is followed:

- latest tag for the most recent stable version
- Unique deployment tags using date and commit ID format

Example:

v20260510-a1b2c3d

This strategy provides:

- Deployment traceability
- Simplified rollback support
- Deployment consistency
- Improved release management

3.2 CONTAINER ORCHESTRATION AND APPLICATION MANAGEMENT

The CaseHub project uses Azure cloud deployment and Docker containerization to manage backend services efficiently. The deployment architecture supports scalability, maintainability, and reliable service execution.

1. Application Deployment Design

Backend Deployment

The Node.js and Express.js backend application is deployed as a containerized cloud service. The deployment setup includes:

- Docker-based backend packaging
- Cloud-hosted backend services
- API-based communication support
- Database connectivity management

Configuration Management

Application configuration values are managed securely through deployment settings and environment variables.

This includes:

- Database connection details
- API configuration settings
- Cloud deployment parameters
- Azure service integration configurations

2. Scalability, Reliability, and Security

Backend Scalability

The application supports scalable backend execution to handle increasing judicial workload and concurrent user requests.

This improves:

- Hearing management efficiency
- Multi-user access handling
- Cloud resource utilization
- Application responsiveness

Deployment Reliability

The deployment process follows a rolling update approach to minimize service interruption during updates.

This ensures:

- Reduced downtime
- Safer deployment updates
- Continuous service availability
- Stable application performance

Security Management

Sensitive application configurations are secured using Azure cloud security mechanisms and protected deployment practices.

Security implementation includes:

- Secure API communication
- Protected deployment credentials
- Authenticated access handling
- Secure database authentication

3.3 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

1. Voice-to-Text Transcription (Azure AI Speech Service)

The system integrates Azure AI Speech Service to provide voice-to-text transcription functionality for courtroom discussions and hearing-related audio.

Implementation

The transcription functionality is integrated through Azure AI Speech Service APIs connected to the backend application.

The service allows:

- Audio-to-text conversion
- Hearing discussion transcription
- Voice note processing
- Cloud-based speech recognition

Process Flow

1. Audio input or courtroom voice recordings are uploaded to the system.
2. The backend sends audio data to Azure AI Speech Service.
3. The service processes the speech input and converts it into text format.
4. Generated transcripts are returned to the application for viewing and storage.

This functionality improves judicial documentation efficiency and reduces manual transcription effort.

2. Legal Document Summarization (Azure AI Language Service)

The project integrates Azure AI Language Service to generate summaries for lengthy legal and judicial documents.

Implementation

The summarization service is integrated through Azure AI Language APIs connected with the backend system.

This service helps:

- Summarize lengthy legal documents
- Reduce manual document review time
- Improve judicial workflow efficiency
- Provide AI-assisted legal insights

Process Flow

1. Legal documents are uploaded or provided as text input.
2. The backend sends document content to Azure AI Language Service.
3. The AI service analyzes the content and generates concise summaries.
4. The summarized output is displayed within the application dashboard.

Benefits of AI Integration

The integration of Azure AI services provides several advantages:

- Faster legal document analysis
- Automated courtroom transcription
- Improved judicial productivity
- Reduced manual workload
- Enhanced courtroom digitalization
- AI-assisted decision support features

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

1.1 DEVSECOPS INTEGRATION

Security is embedded across the development lifecycle through lightweight, fast, and enforceable static checks. Static application security testing is performed via language-native linters on both services: ESLint for the React frontend and Go linters for the backend (e.g., go vet/format and common lint rules). These tools analyse source code for correctness, unsafe patterns, and security hygiene, including unhandled errors, misuse of concurrency primitives, dangerous DOM sinks, and unsafe string handling. Linting runs locally and in continuous integration on every pull request, and merges are blocked when violations are detected to ensure issues are addressed before code reaches production.

Code quality and maintainability are enforced with strict linting configurations that standardise style, imports, and typing conventions. In the frontend, ESLint rulesets (with TypeScript support) prevent common pitfalls such as unused variables, non-exhaustive React hooks dependencies, and weak typing that can mask trust boundary issues. In the backend, Go linters enforce idiomatic patterns, detect shadowed variables, potential nil dereferences, and missing error checks—reducing classes of defects that often lead to security vulnerabilities at runtime.

Container security is strengthened through minimal base images and deterministic builds. The backend is packaged using a Docker multi-stage process that produces a final image based on `scratch`, which removes shells, package managers, and non-essential utilities to reduce the attack surface and the number of potential CVEs. Images are built for the correct target platform (`linux/amd64`) to ensure consistent, reproducible deployments across environments. This approach limits runtime footprint and minimises opportunities for exploitation in production containers.

Operational consistency and secure configuration are maintained through configuration-as-code and environment-driven settings. Sensitive values are injected via environment variables rather than hardcoded, and the backend enforces secure database connectivity (e.g., SSL mode required). CORS is constrained to the exact frontend domain to prevent broad origin access, aligning with the principle of least privilege.

Together, these practices provide practical DevSecOps coverage focused on early defect and vulnerability prevention (via linters), minimized container attack surface (via `scratch` images), and secure-by-default application configuration—delivering measurable security benefits without heavy operational overhead.

Demo:

> **Error:** Security warning: use of env outside the scope Main.go/env/vars

Error: Security warning: use of env outside the scope Products.go/env/vars

Error: Security warning: use of env outside the scope gorm

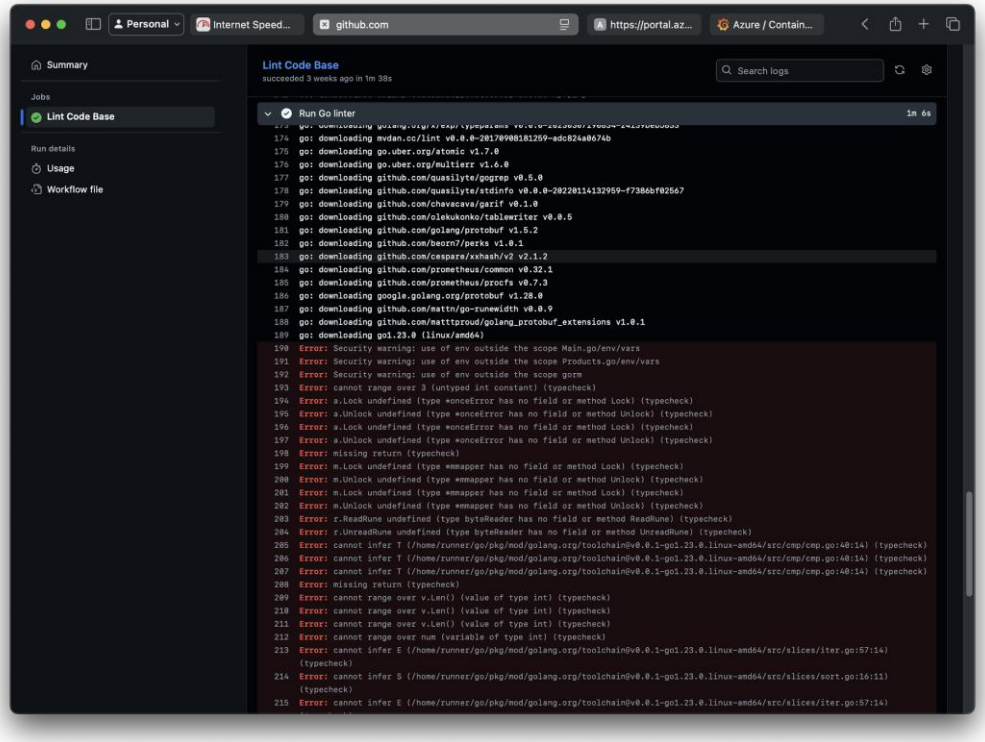


Figure 5. Github Workflow Showing Lint Errors

After: Errors resolved

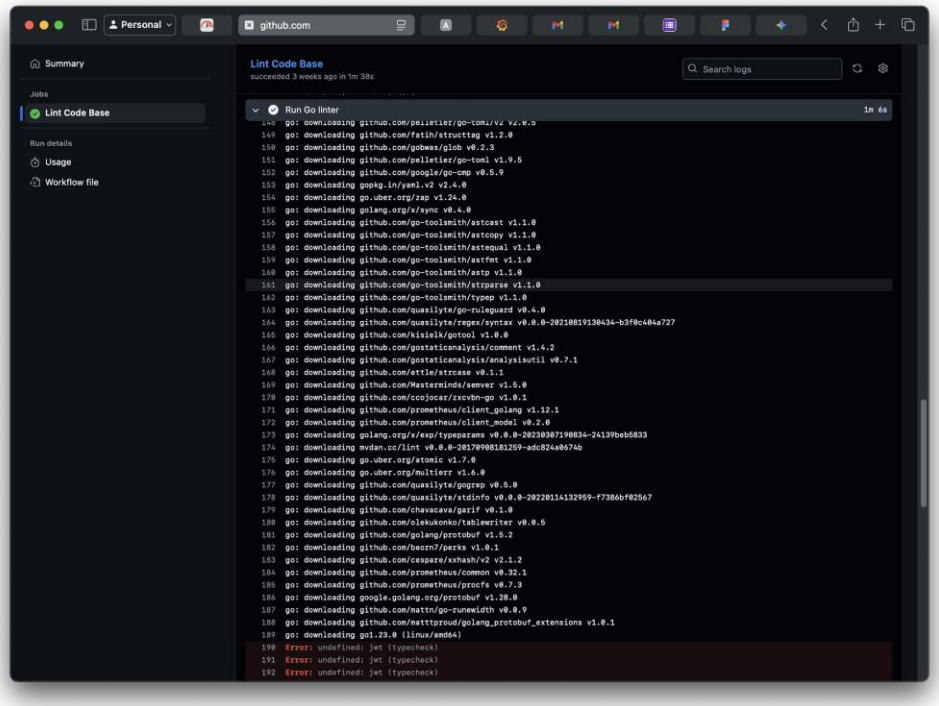


Figure 6. Github Workflow Showing Resolved Env Errors

CHAPTER 4 – CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security practices are integrated throughout the development and deployment lifecycle of the CaseHub platform using lightweight DevSecOps methodologies. The project focuses on secure coding practices, automated validation, secure deployment strategies, and cloud-based protection mechanisms.

Static code analysis and code quality checks are performed during development and CI/CD execution. Frontend validation is handled using standard JavaScript validation and linting practices, while backend validation ensures secure API handling, proper error management, and reliable database communication.

The GitHub Actions CI/CD pipeline automatically validates the application during deployment. Build failures, dependency issues, and deployment errors are identified before production deployment, reducing the possibility of unstable releases.

Container security is enhanced through Docker-based lightweight backend deployment. The backend application is containerized with minimal runtime dependencies to reduce the attack surface and improve deployment consistency.

Sensitive application configurations such as database credentials and deployment settings are managed securely using environment variables and GitHub Secrets instead of hardcoded values.

The backend also enforces secure database communication using authenticated Azure SQL Database connectivity and HTTPS-based API communication.

These DevSecOps practices improve:

- Deployment reliability
- Secure application configuration
- Reduced deployment risks
- Better cloud deployment consistency
- Improved operational security

Figure 5. GitHub Workflow Showing Deployment Validation

(Insert GitHub Actions pipeline screenshot here)

4.2 MONITORING AND OBSERVABILITY

The CaseHub platform utilizes Azure cloud monitoring capabilities to observe application performance, deployment health, and backend service availability.

Application monitoring helps track:

- Backend API availability
- Database connectivity
- Deployment success status

- Application responsiveness
- Cloud resource utilization

GitHub Actions workflows provide deployment logs and CI/CD execution reports that help identify build failures, deployment issues, and configuration problems during application updates.

Azure cloud services provide operational visibility into:

- App Service health
- Azure SQL Database availability
- Backend service status
- Cloud deployment activity

Application logs and backend monitoring help developers troubleshoot issues such as:

- Database connection failures
- API response issues
- Backend deployment problems
- Service interruption events

Monitoring and observability improve:

- System reliability
- Faster issue detection
- Cloud deployment management
- Application maintenance efficiency

4.3 ACCESS CONTROL

Access control mechanisms are implemented at both the cloud infrastructure level and application level to ensure secure judicial data management.

At the Azure cloud level, access permissions are managed through Azure authentication and role-based access mechanisms. Only authorized users and deployment workflows can access application resources and cloud services.

The application itself restricts sensitive operations such as hearing management and room allocation to authenticated judicial users and administrators.

Database access is secured using:

- Azure SQL authentication
- Secure database credentials
- Firewall configuration
- Restricted cloud access rules

Sensitive deployment information such as:

- Database passwords
- Azure deployment credentials
- API configuration values

are securely managed using GitHub Secrets and environment variables.

API communication between frontend and backend services is protected using HTTPS protocols to ensure secure data transmission.

These access control mechanisms improve:

- Data confidentiality
- Secure judicial operations
- Controlled cloud access
- Secure deployment practices

4.4 DEPLOYMENT STRATEGY AND DISASTER RECOVERY PLANNING

The CaseHub platform follows a cloud-based deployment strategy designed to ensure application availability, reliable updates, and simplified recovery procedures.

The deployment process uses GitHub Actions CI/CD workflows integrated with Azure cloud deployment services. Whenever updates are pushed to the GitHub repository, automated build and deployment processes are triggered.

The backend deployment process includes:

- Docker image generation
- Backend build validation
- Azure deployment updates
- Application health verification

The deployment workflow minimizes downtime by validating application health before deployment completion.

Backend health checks ensure:

- API availability
- Database connectivity
- Backend service responsiveness

If deployment issues occur, previous stable Docker image versions can be redeployed quickly using the versioned deployment strategy.

Infrastructure recovery is simplified using Terraform Infrastructure-as-Code configurations. Azure cloud resources can be reprovisioned using Terraform scripts in case of deployment failures or cloud environment issues.

Azure SQL Database provides:

- Automated backups
- Cloud-based data recovery
- Secure data storage
- Backup restoration support

Disaster recovery planning focuses on:

- Rapid deployment restoration

- Secure cloud reprovisioning
- Reliable database recovery
- Minimal service interruption

These practices improve:

- Deployment stability
- Service availability
- Recovery reliability
- Cloud operational resilience

CHAPTER 5 – RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The CaseHub judicial management platform was successfully implemented and deployed on Microsoft Azure cloud infrastructure. The project achieved its primary objectives of courtroom management automation, room allocation handling, cloud deployment, and AI-based judicial assistance.

The frontend application was successfully developed using HTML, CSS, and JavaScript, providing an interactive dashboard for judges and administrators. Backend APIs developed using Node.js and Express.js successfully handled hearing management, room allocation, database operations, and AI service communication.

Azure SQL Database integration was successfully completed for storing structured hearing and judicial records. The backend API established secure communication with the cloud database environment.

Docker containerization successfully packaged the backend services into lightweight and portable deployment units. GitHub Actions automated the CI/CD workflow, enabling continuous deployment and simplified cloud updates.

The Terraform-based infrastructure setup successfully provisioned Azure cloud resources, improving deployment consistency and infrastructure management.

The Azure AI integrations also functioned successfully:

- Azure AI Speech Service converted voice input into text transcripts.
- Azure AI Language Service generated legal document summaries.

The complete cloud architecture successfully demonstrated the integration of:

- Cloud computing
- DevOps practices
- Infrastructure automation
- AI-powered courtroom assistance

The implementation validated the effectiveness of Azure cloud services for building intelligent judicial management platforms.

5.2 CHALLENGES FACED AND RESOLUTIONS

Several technical and deployment-related challenges were encountered during the development of the CaseHub platform.

One major challenge involved establishing secure connectivity between the backend application and Azure SQL Database. Initial firewall and public access restrictions prevented successful database communication. This issue was resolved by properly configuring Azure SQL firewall rules and enabling secure access settings.

Another challenge involved deployment consistency across environments. Docker containerization was implemented to ensure that backend services executed consistently in both development and deployment environments.

CI/CD workflow configuration also presented challenges during automated deployment setup. GitHub Actions workflows were refined to correctly automate build, deployment, and backend update processes.

Resource limitations and quota restrictions in the Azure for Students subscription also affected deployment region selection and cloud resource provisioning. Alternative supported Azure regions were selected to resolve quota limitations.

AI service integration required additional backend configuration and secure API handling. Azure AI Speech and Language services were successfully connected through backend APIs after resolving authentication and deployment issues.

These challenges helped improve understanding of:

- Azure cloud deployment
- Docker containerization
- CI/CD automation
- Cloud networking and security
- AI service integration

5.3 PERFORMANCE AND COST OBSERVATION

Performance Analysis

The implemented CaseHub platform demonstrated stable performance and efficient cloud-based execution.

The backend APIs provided responsive hearing management and room allocation operations with efficient Azure SQL Database communication.

Frontend dashboard pages loaded efficiently due to lightweight frontend implementation and Azure cloud hosting.

Docker containerization improved deployment consistency and simplified backend execution across environments.

Azure AI services also performed efficiently:

- Speech-to-text transcription generated accurate textual outputs from voice input.

- Document summarization reduced lengthy legal content into concise summaries.

The GitHub Actions CI/CD pipeline automated deployment processes successfully and reduced manual deployment effort.

Cost Optimization

The project was designed using Azure for Students subscription resources to minimize operational expenses during academic development.

Cost optimization strategies included:

- Using student subscription credits
- Lightweight backend deployment
- Limited cloud resource allocation
- Managed Azure services
- Efficient containerized deployment

The use of Docker containerization and cloud-managed services reduced deployment overhead and simplified infrastructure management.

Terraform automation also reduced operational complexity and improved deployment efficiency.

The overall architecture successfully demonstrated a cost-effective cloud-native judicial management platform suitable for academic implementation and future scalability.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of the CaseHub platform provided valuable practical experience in cloud computing, DevOps, AI integration, and collaborative software development.

The team gained hands-on experience in:

- Microsoft Azure cloud deployment
- Azure SQL Database integration
- Docker containerization
- Terraform Infrastructure-as-Code
- GitHub Actions CI/CD pipelines
- AI service integration
- Backend API development

One of the major learnings involved understanding real-world cloud deployment workflows and cloud infrastructure management.

The project also improved knowledge in:

- Secure cloud communication
- Database connectivity management
- Automated deployment processes
- Backend container deployment

- Cloud-based AI service usage

The collaborative development process improved teamwork, task coordination, and version control management through GitHub-based collaboration.

Each team member contributed to:

- Frontend development
- Backend API development
- Database integration
- Cloud deployment
- CI/CD workflow configuration
- AI integration
- Documentation and testing

The project significantly strengthened both technical and collaborative development skills.

CHAPTER 6 – CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The CaseHub project successfully delivered an AI-powered cloud-based judicial management platform that automates courtroom operations and integrates modern cloud technologies with intelligent AI services.

The system effectively addressed major courtroom management challenges such as hearing scheduling, room allocation, legal document handling, and judicial workflow management.

The integration of Azure cloud services, Docker containerization, Terraform automation, and CI/CD deployment pipelines demonstrated the practical implementation of modern DevOps and cloud-native technologies.

The AI-powered features including:

- Voice-to-text transcription
- Legal document summarization

enhanced the intelligence and usability of the judicial management system.

The project also demonstrated how cloud computing and Generative AI technologies can improve efficiency, scalability, and automation within judicial environments.

Overall, the project achieved its objectives successfully and provided strong practical exposure to cloud architecture, DevOps practices, backend development, and AI service integration.

6.2 FUTURE WORKS

The current implementation of CaseHub can be further enhanced with several advanced features and intelligent automation capabilities.

Short-Term Future Enhancements

Advanced Authentication System

Future versions can include:

- Role-based access control
- Multi-factor authentication
- Secure judicial login management

Real-Time Notifications

The system can be enhanced with:

- Real-time hearing reminders
- SMS and email notifications
- Cloud-based alert systems

Improved AI Assistance

Additional AI capabilities may include:

- AI-powered legal recommendations
- Smart case classification
- Advanced transcript analytics

Long-Term Future Vision

Full E-Court Integration

The platform can be expanded into a complete digital courtroom management ecosystem including:

- E-filing systems
- Digital evidence management
- Online hearing support

National Judicial Deployment

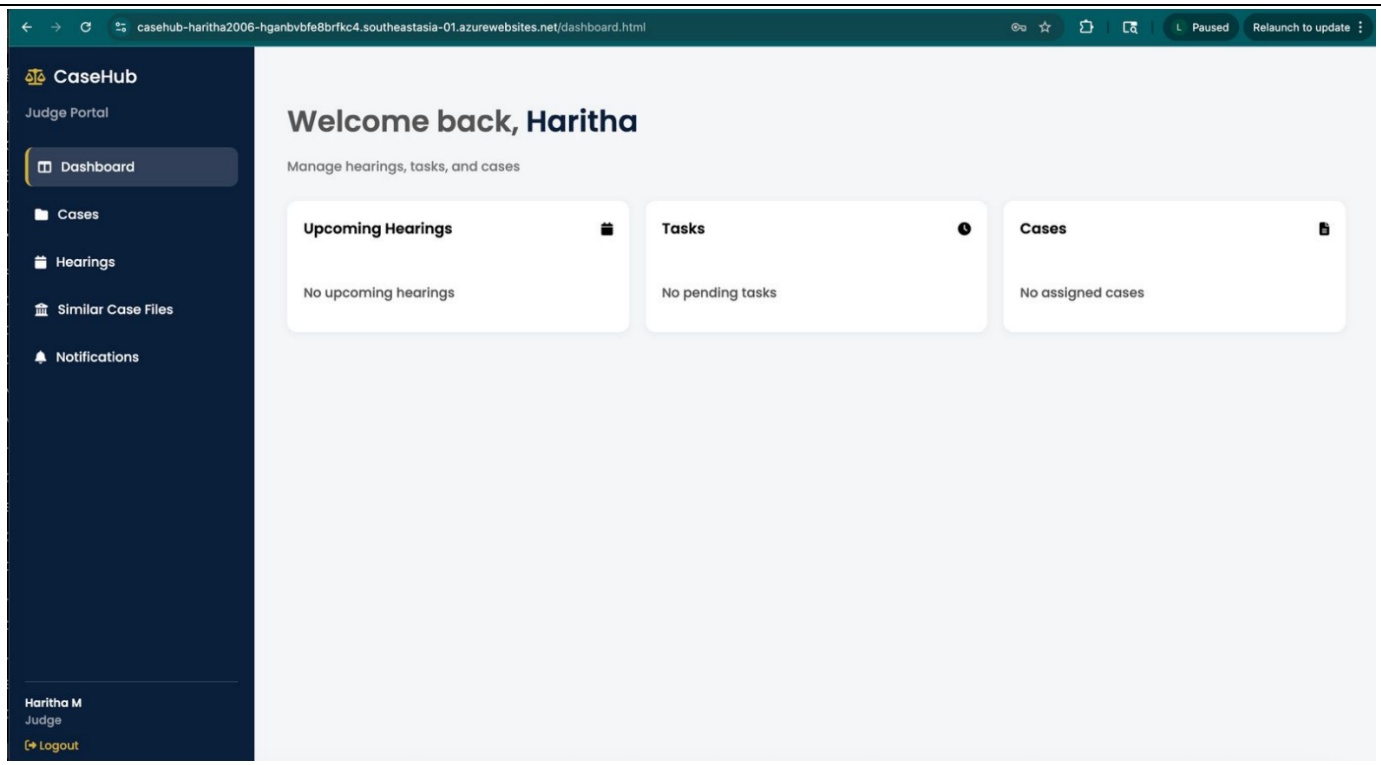
Future deployment can support:

- Multi-court integration
- Scalable nationwide cloud deployment
- Centralized judicial data management

Advanced Analytics Dashboard

AI and cloud analytics can be used to provide:

- Hearing statistics
- Judicial workload analysis
- Predictive courtroom management insights



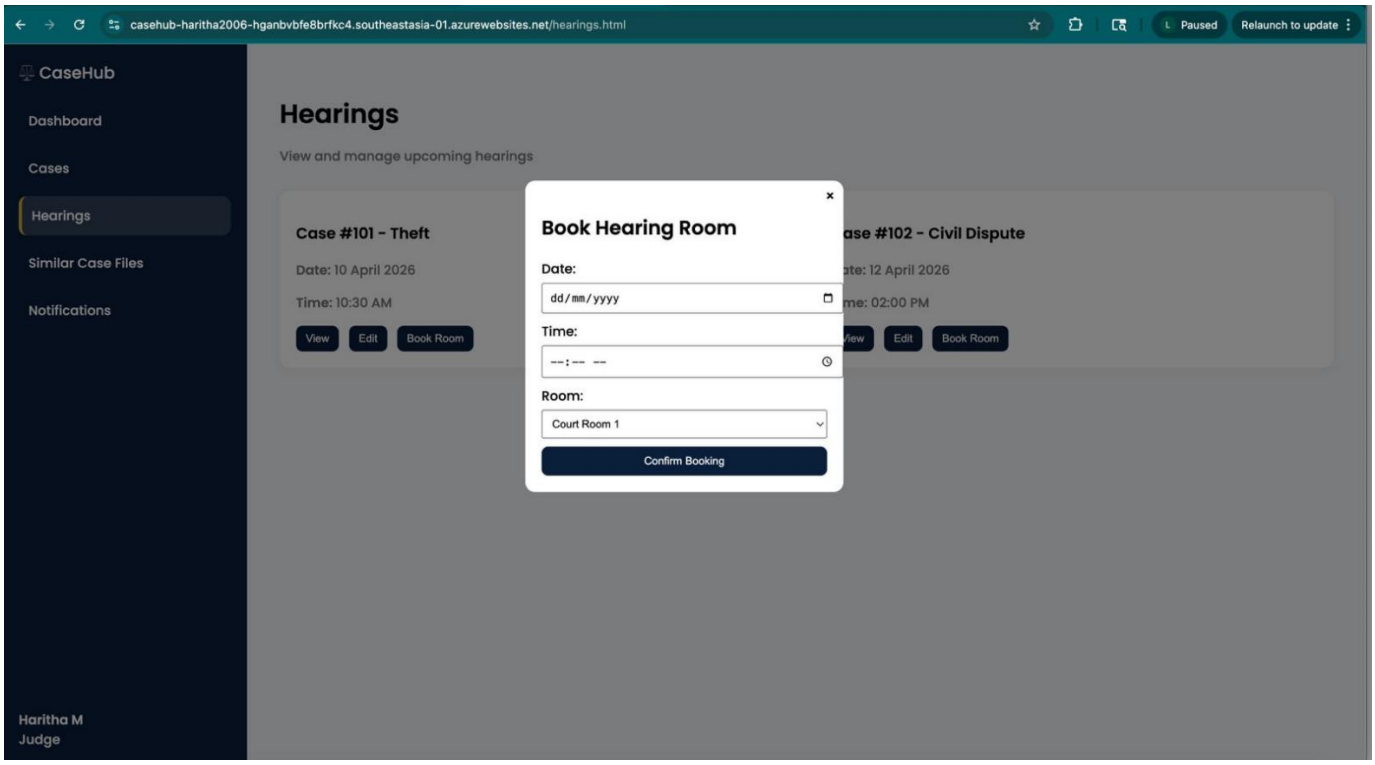
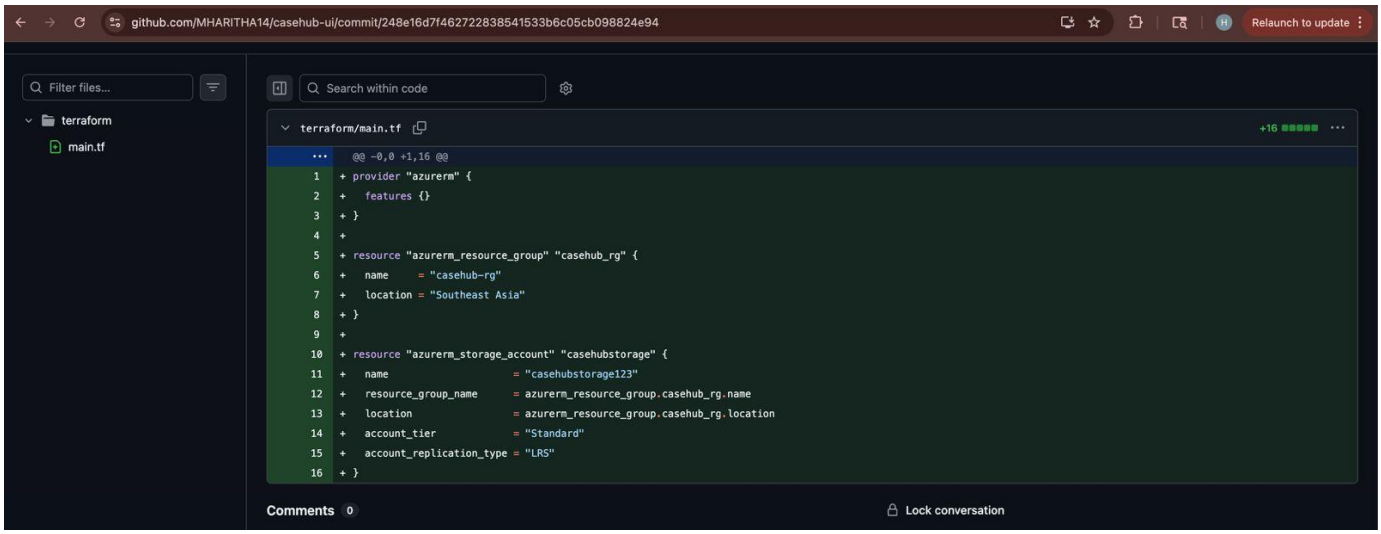


Fig: Output screenshots

APPENDICES

Appendix A – Terraform Code Snippets



Appendix C – Screenshots (Deployments, AI Integration, Security Sca

Successful ci/cd run:

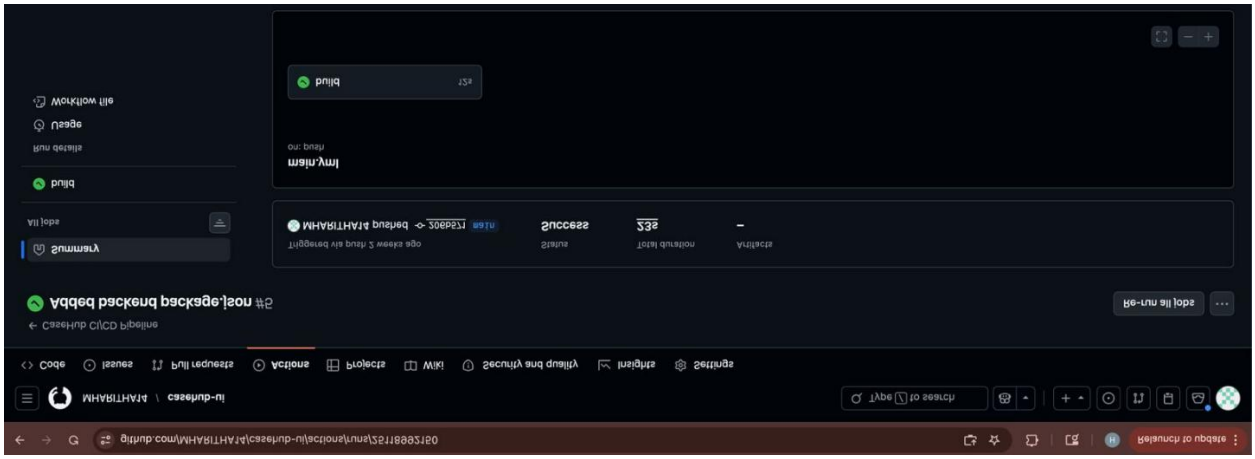


Figure 16. Workflow Showing Frontend and Backend Successfully Deployed

Integrated AI services:

1. **Azure content moderation**

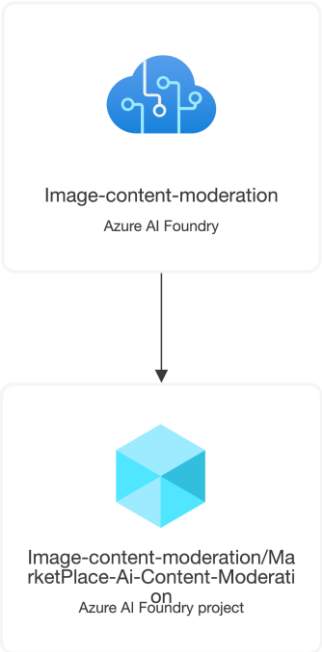


Figure 17. Azure content moderation

1. Azure Container Apps

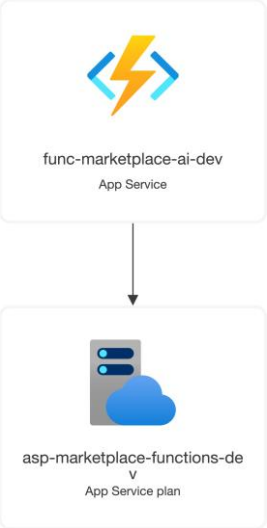
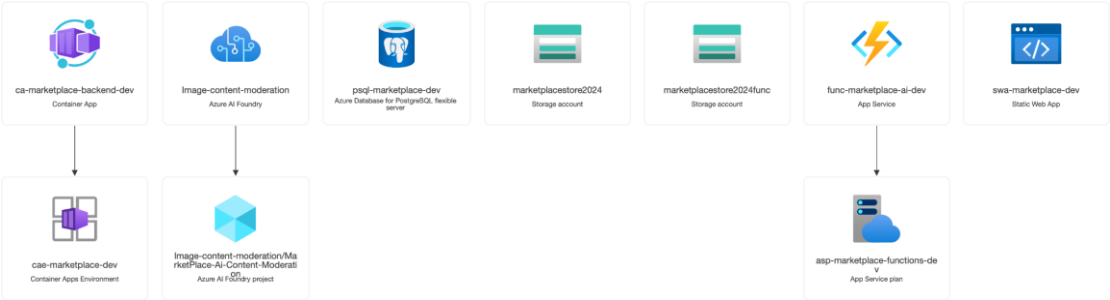


Figure 18. Description Generation with gemini vision pro

Deployment resource visualizer